

Clustered Index in SQL Server with Examples

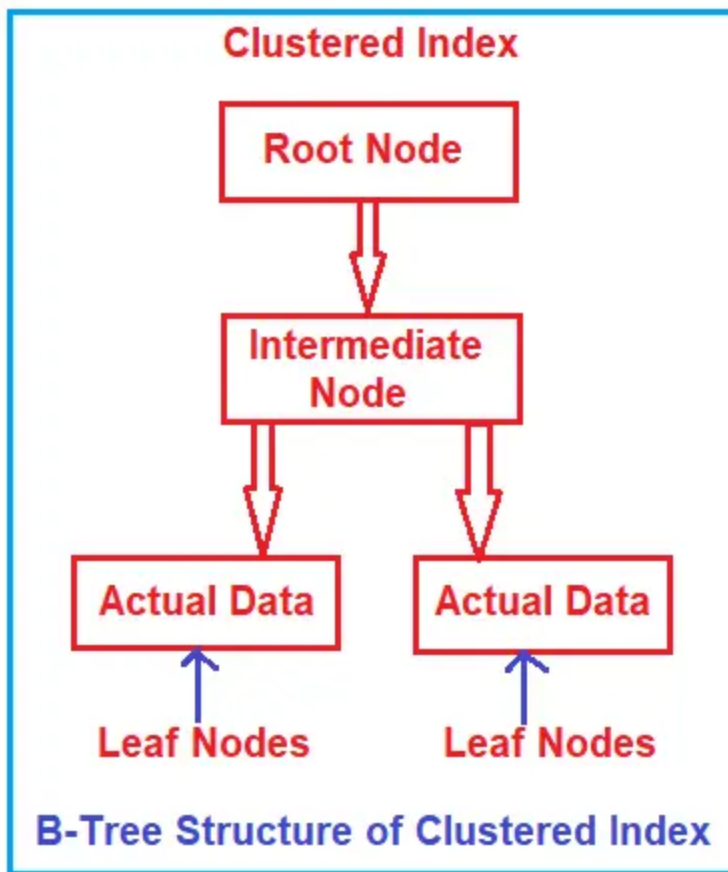
In this article, I am going to discuss the **Clustered Index in SQL Server** with examples. Please read our previous article where we discussed the basic concept of [SQL Server Indexes](#) as well as we also discussed how we can improve the search operations using SQL Server Indexes. As part of this article, we are going to discuss the following pointers.

1. [Understanding the B-Tree structure of Clustered Index](#)
2. [What is a Clustered Index?](#)
3. [Why a table can have only one Clustered Index?](#)
4. [Example to understand the Clustered Index.](#)
5. [Understanding the SQL Server Composite Clustered Index.](#)

As we already discussed in our previous article, whenever we create an index, SQL Server will internally create something called **Balanced Tree (B-Tree) Structure**. As we have two types of Indexes i.e. Clustered and Non-clustered, so, let us first understand the Balanced Tree (B-Tree) Structure for both Clustered and Non-clustered Indexes. In this article, we are going to discuss the Balanced Tree (B-Tree) Structure of Clustered Index and in the next article, we will discuss the [Balanced Tree \(B-Tree\) Structure of the Non-Clustered Index](#) in SQL Server.

Understanding the B-Tree structure of Clustered Index in SQL Server:

Let us understand the Balanced Tree (B-Tree) structure of the clustered index. Please have a look at the following diagram which shows the B-Tree structure of the Clustered Index in SQL Server. As you can see in the below image, the leaf nodes contain the actual data. So, when we search for any data, the clustered index directly gets the data from the leaf node.



What is SQL Server Clustered index?

The Clustered Index in SQL Server defines the order in which the data is physically stored in a table. That is the leaf node (Ref to the B-Tree Structure diagram) stores the actual data. As the leaf nodes store the actual data, a table can have only one clustered index. The Clustered Index by default was created when we created the primary key constraint for that table. That means the primary key column creates a clustered index by default.

When a table has a clustered index then the table is called a clustered table. If a table has no clustered index, its data rows are stored in an unordered structure.

Understanding the Clustered Index with an example in SQL Server:

In order to understand the clustered index and to make sure that the clustered index by default was created when we created the primary key constraint on a table, please execute the following query.

```
CREATE TABLE Employee
(
  Id INT PRIMARY KEY,
  Name VARCHAR(50),
  Salary INT,
  Gender VARCHAR(10),
  City VARCHAR(50),
  Dept VARCHAR(50)
)
```

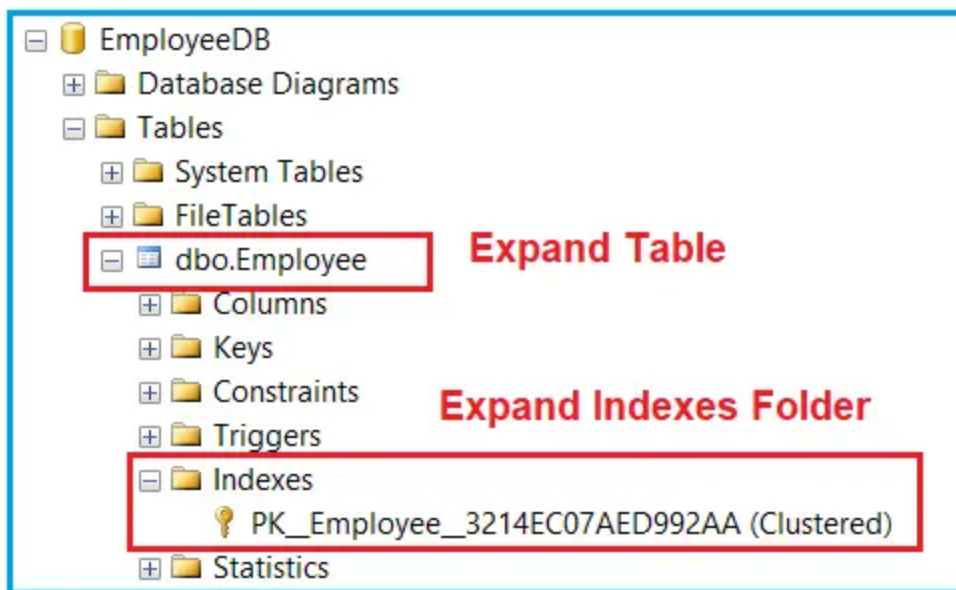
As you can see in the above query, we created the Employee table having the Id column as the Primary key column. The Primary key constraints in SQL Server by default create clustered indexes if there is no clustered index that already exists on the table. To Confirm whether the Clustered index is created or not execute any one of the following statements.

sp_helpindex Employee
Execute sp_helpindex Employee

Once you execute any of the above SQL statements, it will give you the following output. Here, you can find the index name, the type of index as index_description, and the column on which the index is created as index_keys.

index_name	index_description	index_keys
PK_Employee_3214EC07AED992AA	clustered, unique, primary key located on PRIMARY	Id

Or alternatively, you can check the below folder.



Now insert some records in random order into the Employee table. Notice the values of the Id column of the Employee table is not in sequential order.

```
INSERT INTO Employee VALUES (3,'Pranaya', 4500, 'Male', 'New York', 'IT')
INSERT INTO Employee VALUES (1,'Anurag', 2500, 'Male', 'London', 'IT')
INSERT INTO Employee VALUES (4,'Priyanka', 5500, 'Female', 'Tokio', 'HR')
INSERT INTO Employee VALUES (5,'Sambit', 3000, 'Male', 'Toronto', 'IT')
INSERT INTO Employee VALUES (7,'Preety', 6500, 'Female', 'Mumbai', 'HR')
INSERT INTO Employee VALUES (6,'Tarun', 4000, 'Male', 'Delhi', 'IT')
INSERT INTO Employee VALUES (2,'Hina', 500, 'Female', 'Sydney', 'HR')
```

Now Fetch the data by executing the below query.

SELECT * FROM Employee;

When you execute the above Select query, it will give you the below output.

Id	Name	Salary	Gender	City	Dept
1	Anurag	2500	Male	London	IT
2	Hina	500	Female	Sydney	HR
3	Pranaya	4500	Male	New York	IT
4	Priyanka	5500	Female	Tokiyo	HR
5	Sambit	3000	Male	Toronto	IT
6	Tarun	4000	Male	Delhi	IT
7	Preety	6500	Female	Mumbai	HR

Notice, the records are sorted in the table in ascending order based on the Id column. This is because of the Clustered Index which we created on the Id column. The job of Clustered Index in the SQL Server is to determine or organized data in either ascending or descending order on the column(s) on which it is created. This also proves that the Clustered Index defines the order in which the data is physically stored in a table. And this is also the reason that it does not allows the creation of more than one clustered index in a table in SQL Server.

Can we create multiple clustered indexes in a table in SQL Server?

It is not possible to create more than one clustered index in a table in SQL Server. The Employee table that we have just created already has one clustered Index on the Id column which already determines the physical storage order of the data in the table. Let us try to create another clustered Index on the Salary column and see what happens. Let us try to execute the following SQL Query.

```
CREATE CLUSTERED INDEX IX_Employee_Salary ON Employee(Salary)
```

When we try to execute the above SQL query, it throws the following error. This is because we already have a clustered index that already arranged the data in the table. This is the reason why a table can have only one clustered index. If you want to create a clustered index, first delete the existing one and then create a new one.

Cannot create more than one clustered index on table 'Employee'. Drop the existing clustered index 'PK__Employee__3214EC07DCDC2006' before creating another.

What is Composite Clustered Index in SQL Server?

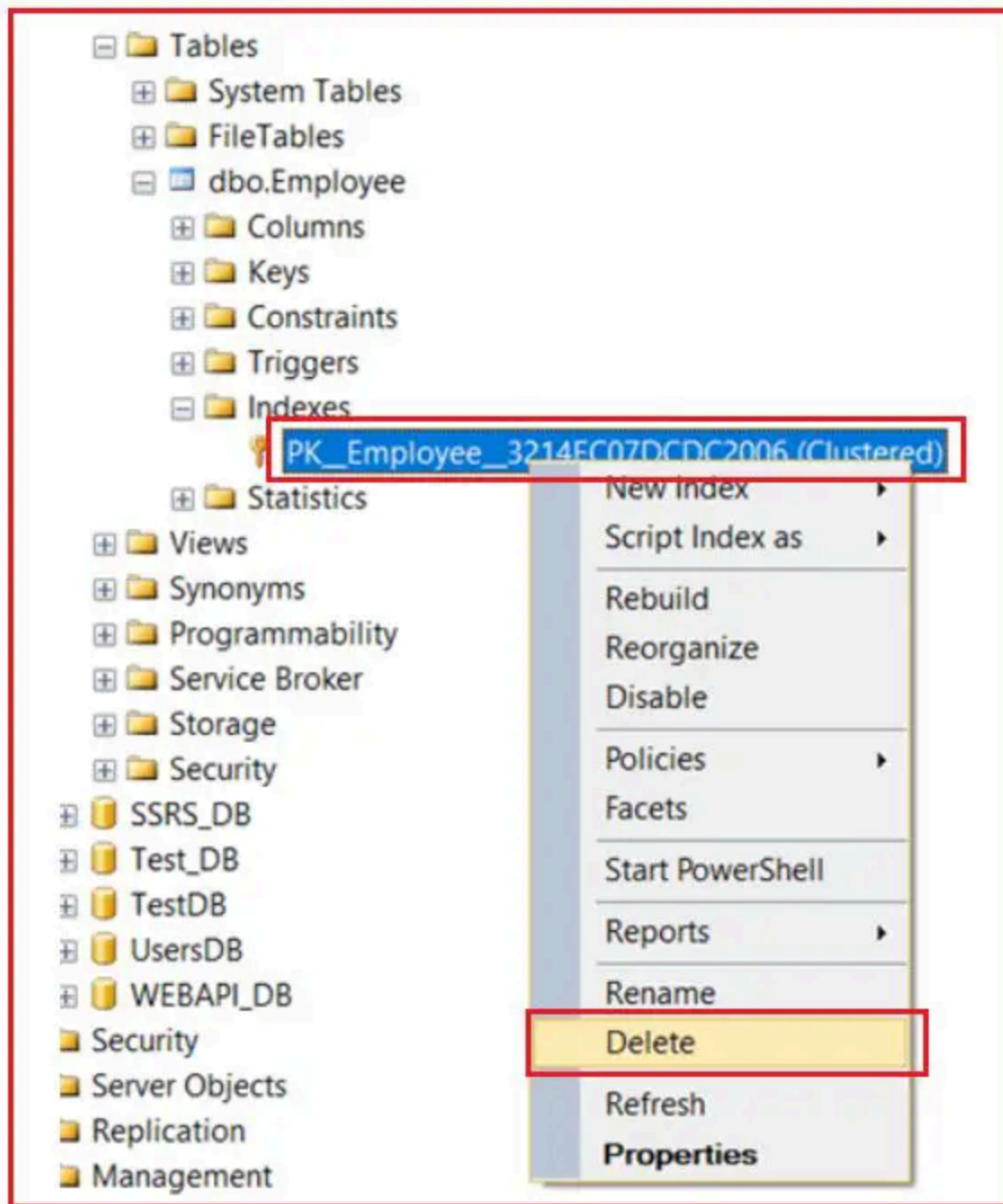
It is possible in SQL Server to create the clustered index with multiple columns and if we do so, then it is called a composite clustered index. Let's create a clustered index based on 2 columns. To do this we first have to drop the existing clustered index which is created on the Id column.

```
DROP INDEX Employee.PK__Employee__3214EC07AED992AA
```

When you execute the above DROP INDEX query, then you will get an error message stating **An explicit DROP INDEX is not allowed on index 'Employee.PK__Employee__3214EC07DCDC2006'. It is being used for PRIMARY KEY constraint enforcement.**

Why it is not allowing that we will discuss in a later article. But for now to delete the clustered index, right-click on the index in the Object Explorer window and select the DELETE option from the context menu as

shown in the below image.



Once you delete the existing Clustered Index, then execute the following CREATE INDEX query to create a composite clustered Index on the Gender and Salary columns of the Employee table.

CREATE CLUSTERED INDEX IX_Employee_Gender_Salary ON Employee(Gender DESC, Salary ASC)

Fetching the Data:

Now issue the following select query to fetch the data,

SELECT * FROM Employee;

When you execute the above select query, you will get the following data. Now, you can see the data FIRST arranged by Gender in descending order and then by Salary in ascending order.

Id	Name	Salary	Gender	City	Dept
1	Anurag	2500	Male	London	IT
5	Sambit	3000	Male	Toronto	IT
6	Tarun	4000	Male	Delhi	IT
3	Pranaya	4500	Male	New York	IT
2	Hina	500	Female	Sydney	HR
4	Priyanka	5500	Female	Tokiyo	HR
7	Preety	6500	Female	Mumbai	HR

In the next article, I am going to discuss the [Non-Clustered Index in SQL Server](#) with Examples. Here, in this article, I try to explain the **Clustered Index in SQL Servers** with Examples. I hope you enjoy this article and understand How Clustered Index works in SQL Servers with Examples.